KRISHICONNECT – PROJECT PROGRESS

An Intelligent Platform for Agricultural Trade, Knowledge Sharing, and Market Insights

1. Project Overview – Krishi Connect

Krishi Connect is an intelligent digital agricultural platform designed to integrate agricultural trading, farmer–buyer connectivity, market information, price forecasting, knowledge sharing, and personalized recommendations into a single platform.

The main objective is to help farmers find suitable buyers, list and sell agricultural products, access market-price trends, understand future price estimates, share agricultural knowledge, and receive relevant recommendations. The platform also includes features such as ratings and reviews, notifications, location-based discovery, QR-based product traceability, and dashboards to improve transparency and decision-making.

1.1 Problems and Solutions – Krishi Connect

Problem	Proposed Solution in Krishi Connect
1. Difficulty finding suitable buyers	Provide an agricultural marketplace where farmers can list their products and buyers can search and identify relevant products.
2. Farmers lack access to reliable market-price information	Develop a Market Information module showing available agricultural prices, historical trends, and market information.
3. Difficulty understanding future price trends	Use ARIMA / Prophet time-series models to generate future agricultural price estimates from historical data.
4. Agricultural information is scattered across different sources	Create a centralized knowledge-sharing community where farmers can ask questions, share experiences, and participate in discussions.
5. Difficulty finding relevant agricultural products or information	Develop a recommendation system using user, product, and activity information to provide relevant suggestions.
6. Limited connection between farmers and buyers	Implement farmer-buyer matching using available product and location information.
7. Lack of trust between marketplace participants	Introduce ratings and reviews so users can provide feedback after transactions.
8. Difficulty finding nearby sellers or products	Use OpenStreetMap and Leaflet to provide location-based product and seller discovery.
9. Farmers may miss important updates	Provide notifications for orders, price alerts, community interactions, recommendations, and other important activities.
10. Limited product traceability	Use QR codes to connect agricultural products with their digital information and origin/listing details.
11. Large and complex agricultural datasets	Use Pandas and NumPy for data cleaning, transformation, preprocessing, and numerical analysis.

12. Need for data-driven decision support	Use Scikit-learn, forecasting models, and visualization tools to generate recommendations, forecasts, and market insights.
13. Managing many types of users and permissions	Implement JWT authentication and role-based access control for Farmers, Buyers, Sellers, and Administrators.
14. Difficulty managing marketplace data	Use PostgreSQL to store structured information such as users, products, orders, market prices, reviews, forecasts, and notifications.
15. Difficulty understanding analytical results	Use Chart.js / Recharts / Plotly to present prices, trends, forecasts, and KPIs through interactive visualizations.
16. API and frontend integration issues	Use REST APIs through the backend and Postman to test and validate API requests independently.
17. Development environment differences	Use Docker to package the application and its dependencies consistently across development and deployment environments.
18. Difficulty maintaining project code	Use Git and GitHub to track changes, manage branches, collaborate, and maintain project history.
19. Application is available only on the local system	Use cloud deployment so the application can be accessed remotely for demonstrations and future users.

What Has Been Completed So Far

- The project problem statement and objectives have been defined. The main problem identified is that farmers often face difficulty finding suitable buyers, accessing reliable market-price information, and getting useful agricultural information at the right time.
- The scope of KrishiConnect has been defined as an intelligent platform combining agricultural trade, market information, knowledge sharing, and decision-support features.
- A literature review has been completed covering 15 research papers related to agricultural marketplaces, farmer-buyer connectivity, agricultural e-commerce, price forecasting, digital agriculture platforms, knowledge sharing, and AI-based market intelligence.
- The methodology, technologies, findings, limitations, relevance, and research gaps of the 15 papers have been documented.
- The major research gap has been identified: existing solutions often address individual agricultural needs separately rather than providing one integrated platform.
- The proposed modules have been identified: farmer-buyer marketplace, market information, price forecasting, knowledge sharing, recommendations, ratings/reviews, notifications, and QR-based traceability.
- Initial technology requirements and the overall direction for data analysis, machine learning, backend development, database management, and application development have been identified.

Work Currently Being Done

- The current work is focused on converting the research findings into a practical system design for KrishiConnect.

- Functional requirements and the workflow of the farmer, buyer, marketplace, market-information, forecasting, knowledge-sharing, and recommendation modules are being finalized.
- Suitable datasets and data sources for agricultural products, market prices, and forecasting are being studied and prepared.
- The system architecture and database structure are being planned so that the different modules can work together.
- The price-forecasting approach is being planned, including data preprocessing, feature selection, model training, evaluation, and integration with the platform.
- The user interface is being planned with an emphasis on keeping the platform simple and understandable for farmers.

2. Tools and Technologies Used / Planned

Layer / Area	Technology / Tool	Main Purpose
Frontend	React.js / Next.js	Build interactive web applications and dashboards
UI	HTML5, CSS3, JavaScript	Page structure, styling and client-side behavior
UI Framework	Tailwind CSS / Bootstrap	Responsive and reusable interface design
Backend	Python FastAPI	REST APIs, business logic and application services
Database	PostgreSQL	Store users, products, orders, discussions and market data
Data Processing	Pandas	Clean, transform and analyze agricultural datasets
Numerical Processing	NumPy	Numerical and mathematical operations
Machine Learning	Scikit-learn	Recommendation and data-driven intelligent features
Forecasting	ARIMA / Prophet	Time-series agricultural price forecasting
Visualization	Chart.js / Recharts	Graphs, charts and analytics dashboards
Authentication	JWT	Secure login and role-based API access
Image Storage	Cloudinary / Firebase Storage	Store product images and return image URLs
Maps	OpenStreetMap + Leaflet	Map display and location-based discovery
QR	QR Code Library	Generate and scan product traceability codes
Notifications	Firebase Cloud Messaging / Email	Send application alerts and updates
API Testing	Postman	Test and validate backend APIs
Version Control	Git + GitHub	Source-code versioning and collaboration
Containerization	Docker	Package frontend/backend/services consistently
Deployment	Cloud platform	Host the application for real users

3.Features Used in the Project

3.1 User Registration and Login

- User registration with required profile information.
- Secure password handling on the server side.
- Login and logout.
- JWT-based authenticated sessions/API access.
- Role-based access for Farmer, Buyer, Seller and Administrator.

3.2 Role-Based Access Control

- Farmer can manage agricultural listings and participate in the community.

- Buyers can search products, place purchase requests/orders and provide reviews.
- Seller can manage product listings and respond to orders.
- Administrators can manage users, products, orders, community content and reports.

3.3 Product Marketplace

- Create product listings.
- Add product name, category, price, quantity and location.
- Upload product images.
- Edit and delete listings.
- View product details.
- Search and filter listings.
- Compare available products.
- Contact or initiate a purchase request with sellers.

3.4 Order Management

- Buyer creates a purchase request/order.
- Seller views incoming orders.
- Seller updates order status.
- Buyer can view order status.
- System creates notifications for important status changes.
- Order history is retained for dashboard and reporting.

3.5 Ratings and Reviews

- Buyer can rate completed transactions.
- Users can provide written feedback.
- Ratings can help other users evaluate marketplace participants.
- Administrators can monitor inappropriate content.

3.6 Knowledge-Sharing Community

- Farmers can ask questions.
- Users can answer or comment.
- Users can share farming experiences.
- Discussions can be organized by topics or categories.
- Users can discover useful discussions through search and recommendations.

3.7 Price Forecasting

- Prepare historical price data.
- Clean and transform time-series records.
- Train/use ARIMA or Prophet models.
- Generate future price estimates.
- Display forecast values together with historical values.
- Store model name and forecast date for traceability.
- Evaluate forecast quality using suitable error metrics before relying on a model.

3.8 Recommendation System

- Collect permitted user activity such as searches, product views and purchases.
- Create user/item features.
- Calculate relevance or similarity.
- Recommend products.
- Recommend sellers/buyers where the use case permits.
- Recommend useful community discussions.
- Improve recommendations as more activity data becomes available.

3.9 Location-Based Search

- Store seller/product location.
- Display relevant locations on a map.
- Allow users to search for nearby products or sellers.
- Support location-based filtering.
- Use OpenStreetMap and Leaflet for map visualization.

3.10 QR Product Traceability

- Generate a unique identifier for a product/listing.
- Create a QR code linked to that identifier.
- Allow the buyer to scan the QR code.
- Display product information and origin/listing details.
- Use traceability to improve transparency.

3.11 Market Price Dashboard

- Display historical commodity prices.
- Filter by commodity and market where data is available.
- Display price trends through charts.
- Compare price movement over selected periods.
- Provide a centralized view of available market information.

3.12 Notifications

- Order created/accepted/rejected notifications.
- Price alert notifications.
- Community reply to notifications.
- Recommendation notifications.
- Administrative announcements.
- Read/unread notification state.

3.13 Personalized Dashboard

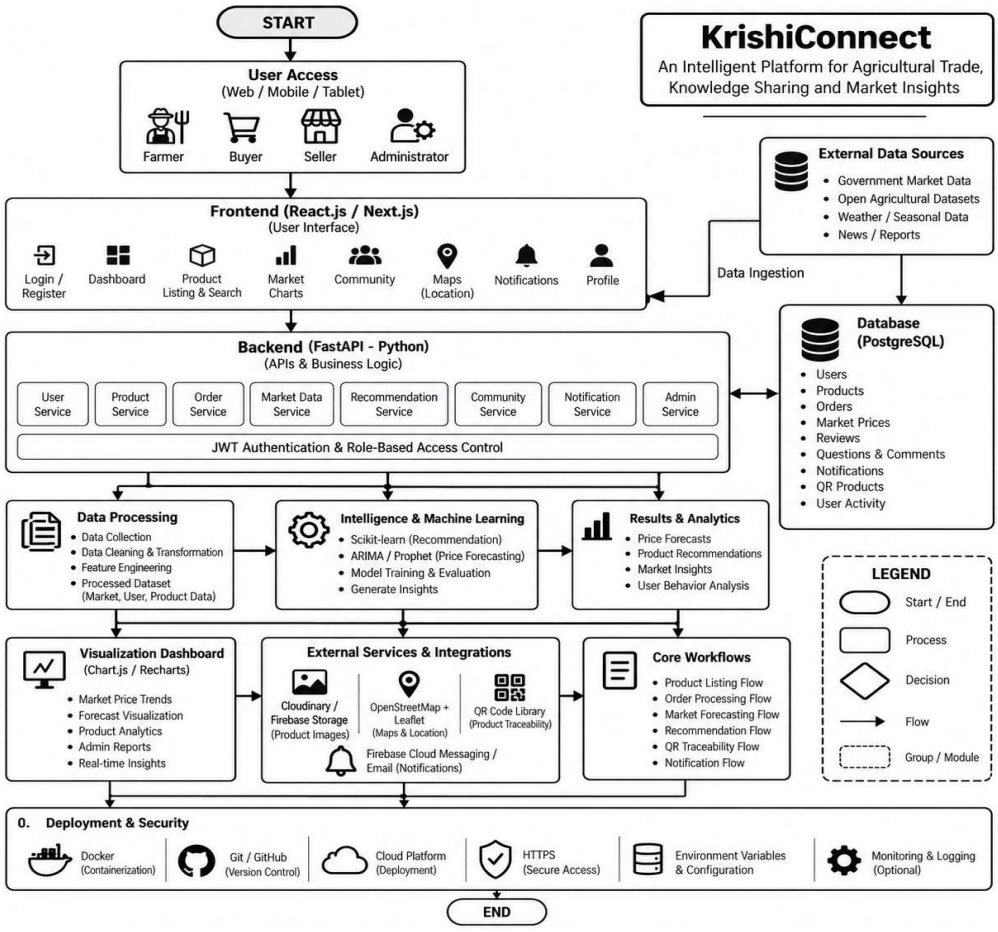
- User profile summary.

- My products.
- My orders.
- Market insights.
- Recommendations.
- Community activity.
- Notifications.
- Relevant analytics.

3.14 Admin Panel

- Manage users.
- Manage product listings.
- Monitor orders.
- Moderate community content.
- View reports and analytics.
- Monitor suspicious or inappropriate activity.
- Manage selected market data operations.

Architecture Diagram – Krishi Connect



3. Next Plan and Remaining Tasks

- Finalize functional and non-functional requirements.
- Finalize system architecture, database schema, and module workflows.
- Collect, clean, and prepare market-price and agricultural datasets.
- Perform exploratory data analysis and identify useful features for forecasting.
- Develop farmer and buyer registration, login, and profile modules.
- Develop product listing, product search, filtering, and buyer-contact functionality.
- Implement farmer-buyer matching.
- Develop the market-information and price-trend module.
- Train, compare, and evaluate suitable price-forecasting models.
- Integrate the selected forecasting model into the application.
- Develop the farmer knowledge-sharing/community module.
- Implement personalized recommendations.
- Add ratings, reviews, and notification features.
- Develop QR-based traceability where the required product information is available.
- Integrate all modules and perform functional, usability, and model-performance testing.
- Fix issues identified during testing and improve the system based on feedback.
- Complete deployment, documentation, final report, presentation, and project demonstration.

4. Planned Features and Modules

- Farmer and Buyer Module – registration, login, profiles, and role-based access.
- Agricultural Marketplace – farmer product listings, buyer search, product details, and communication.
- Market Information – market prices, trends, and relevant market information.
- Price Forecasting – machine-learning/time-series prediction of future agricultural commodity prices.
- Buyer Matching – matching farmers with relevant buyers using available product and location information.
- Knowledge Sharing – farmer questions, experiences, discussions, and useful agricultural information.
- Recommendation System – personalized suggestions based on available user, product, and market information.
- Ratings and Reviews – feedback and trust-building between users.
- Notifications – important updates about listings, buyers, market information, and platform activity.
- QR Traceability – QR-based access to product-related information and traceability data.

5. Expected Plan for Completing the Project

- Phase 1 – Requirement Finalization: finalize requirements, users, use cases, workflows, and database requirements.
- Phase 2 – Data Preparation: collect data, clean it, perform exploratory analysis, and prepare forecasting features.
- Phase 3 – System Design: finalize architecture, database, APIs, module workflows, and UI design.
- Phase 4 – Core Development: develop authentication, profiles, product listings, search, filtering, and marketplace functions.
- Phase 5 – Intelligent Modules: develop market analysis, price forecasting, buyer matching, and recommendation components.
- Phase 6 – Knowledge and Trust Features: add knowledge sharing, ratings, reviews, notifications, and QR traceability.

- Phase 7 – Integration and Testing: connect all modules, test workflows and APIs, evaluate the forecasting model, and fix problems.
- Phase 8 – Deployment and Finalization: deploy the prototype, prepare documentation, complete the final report, and conduct the final demonstration

Area	Recommended Choice	Reason
Frontend	React.js	Simple component-based UI and large ecosystem
Backend	FastAPI	Python-based API layer and easy ML integration
Database	PostgreSQL	Reliable relational database for marketplace data
Data	Pandas + NumPy	Clean and process agricultural datasets
ML	Scikit-learn	Recommendation/intelligent data-processing features
Forecasting	ARIMA / Prophet	Agricultural time-series price forecasting
Charts	Recharts / Chart.js	Interactive market and admin dashboards
Authentication	JWT	Role-aware protected API access
Images	Cloudinary	External image storage and delivery
Maps	OpenStreetMap + Leaflet	Location-based marketplace
QR	QR Code library	Product traceability
Notifications	Firebase Cloud Messaging	Application alerts
Testing	Postman	API verification
Version Control	Git + GitHub	Source-code management
Deployment	Docker + Cloud	Repeatable deployment

1.1 React.js / Next.js — Frontend

React.js is used to build the interactive and user-friendly interface of KrishiConnect. Since the platform has different sections such as login, product listings, orders, community discussions, market analytics, recommendations, notifications, and administration, React helps us develop these sections as reusable components.

Next.js can be used along with React when we need features such as better routing, optimization, and a more production-ready application structure.

Why we chose it:

- Components can be reused across different pages, reducing duplicate code.
- Dashboards and other sections can update dynamically without refreshing the entire page.
- Search, filters, forms, and charts can easily communicate with the backend.
- Component-based structure makes the application easier to maintain.
- Next.js provides built-in routing and additional optimization features.
- It supports responsive interfaces for different screen sizes.

1.2 HTML5, CSS3 and JavaScript — Web Foundation

HTML5, CSS3, and JavaScript form the foundation of our web application. HTML5 is responsible for the structure of the pages, CSS3 handles the visual appearance and layout, while JavaScript adds interaction and dynamic behavior.

These technologies work underneath React/Next.js to create a complete web interface.

Why we chose them:

- HTML helps us create forms, tables, cards, menus, and other page elements.
- CSS helps design the layout, spacing, fonts, and responsive interface.
- JavaScript handles user interactions and browser-side operations.
- Together, they provide the foundation required for React components.

1.3 Tailwind CSS / Bootstrap — User Interface

Tailwind CSS or Bootstrap can be used to make the KrishiConnect interface consistent, responsive, and easier to develop. Instead of creating every design element from the beginning, these frameworks provide reusable styling utilities and components.

Why we chose it:

- Speeds up the development of the user interface.
- Helps the application work properly on mobile, tablet, and desktop.
- Maintains a consistent design throughout the platform.
- Provides reusable styling classes and components.
- Particularly useful for dashboards, forms, cards, and marketplace pages.

1.4 Python FastAPI — Backend

Fast API can be used to build the backend APIs of Krishi Connect. One important reason for considering Fast API is that our project also uses Python for data processing, forecasting, and machine-learning features.

The backend acts as a bridge between the frontend and different services such as products, orders, users, market information, recommendations, and notifications.

Why we chose it:

- Python works directly with Pandas, NumPy, and machine-learning libraries.
- REST APIs keep the frontend and backend separate and organized.
- Automatic API documentation makes development and testing easier.
- Built-in request validation helps handle incorrect inputs.
- It supports asynchronous operations where required.
- It is suitable for developing modular backend services.

1.5 PostgreSQL — Relational Database

PostgreSQL is used as the main database for structured Krishi Connect data. The project contains many connected entities, for example, users create products, buyers place orders, products receive reviews, and market prices are associated with commodities and dates.

Because these relationships are important, a relational database is suitable for the project.
Why we chose it:

- Supports relationships between tables using foreign keys.
- Provides reliable transactions for operations such as orders.
- Supports complex queries for searching and reporting.
- Works well with structured marketplace and agricultural data.
- Indexing can improve data retrieval speed.
- Provides room for the system to grow beyond a basic prototype.

1.6 Pandas — Data Processing

Pandas is used for working with agricultural and market-price datasets. Before we use the data for analysis or forecasting, it needs to be cleaned, organized, and transformed.

Pandas makes this process much easier, especially when the data comes from CSV files or databases.
Why we chose it:

- Helps identify and handle missing values.
- Makes it easier to work with date and time information.
- Allows filtering based on crops, markets, and locations.
- Helps group data by commodity and time period.
- Makes it easy to calculate averages, minimums, maximums, and trends.
- Prepares clean datasets for machine-learning and forecasting models.

1.7 NumPy — Numerical Processing

NumPy is used for numerical calculations and data preparation. It works closely with Pandas and Scikit-learn and provides efficient operations on numerical data.
Why we chose it:

- Supports array and matrix operations.
- Makes mathematical calculations easier.
- Helps with numerical preprocessing.
- Provides the numerical foundation required by several Python data-science libraries.

1.8 Scikit-learn — Intelligent / Recommendation Layer

Scikit-learn is planned for the machine-learning components of KrishiConnect, particularly recommendation and other intelligent features.

For example, user activity can be converted into useful features and then used to identify products, discussions, or other content that may be relevant to the user.
Why we chose it:

- Provides tools for preprocessing and feature engineering.
- Supports classification, clustering, and similarity-based algorithms.
- Useful for developing an initial recommendation system.
- Works smoothly with Pandas and NumPy.
- Makes it easier to experiment with and evaluate different models.

1.9 ARIMA / Prophet — Price Forecasting

ARIMA and Prophet are considered for agricultural price forecasting. They can use historical price information to identify patterns and generate estimates for future prices.

These predictions will be presented as estimates rather than guaranteed future prices.
Why we chose them:

- They are designed for time-series data.
- They can identify trends and patterns in historical prices.
- They can generate future price estimates.
- Forecast results can be displayed through charts.
- Different models can be evaluated using suitable performance metrics before choosing one for the final system.

1.10 Chart.js / Recharts — Visualization

Chart.js or Recharts can be used to turn numerical agricultural data into easy-to-understand visual information.

For example, farmers can view historical price movements and forecast values through charts instead of having to interpret large tables of numbers.
Why we chose them:

- Line charts can show price trends over time.
- Bar charts can compare products or categories.
- Historical and forecast values can be displayed together.
- Useful for dashboards and analytical reports.
- Makes complex numerical information easier to understand.

1.11 JWT — Authentication and Authorization

JWT is used for secure user authentication and role-based access. After a user logs in, the system can use the token to verify the user's identity when accessing protected features.

This is particularly useful because KrishiConnect has different roles such as Farmer, Buyer, Seller, and Administrator.

Why we chose it:

- Separates public APIs from protected APIs.
- Supports role-based access control.
- Helps protect product, order, profile, and administrative APIs.
- Allows the backend to verify whether a user is authorized to perform an operation.

1.12 Cloudinary / Firebase Storage — Image Storage

Agricultural products may require several images, so storing large image files directly inside PostgreSQL is not ideal. Cloudinary or Firebase Storage can be used to store these images, while the database stores their URLs and related information.

Why we chose it:

- Provides dedicated storage for product images.
- Can improve image delivery and loading.
- Keeps PostgreSQL focused on structured information.
- Suitable for storing photos of agricultural products and listings.

1.13 OpenStreetMap + Leaflet — Location Features

OpenStreetMap and Leaflet can be used to provide location-based features in KrishiConnect. For example, users could view the locations of sellers or search for products available near them.

Why we chose it:

- Allows product and seller locations to be displayed on maps.
- Supports nearby product and seller discovery.
- Enables location-based filtering.
- Provides an interactive map without developing a mapping system from scratch.

1.14 QR Code Library — Product Traceability

QR code can connect a physical agricultural product or listing with its digital information. Each product can have a unique identifier that is converted into a QR code.

When a buyer scans the code, the associated product information can be displayed.

Why we chose it:

- Generates unique QR codes for products.
- Allows buyers to access product information through scanning.
- Can display product origin and listing details.

- Improves transparency and traceability.

1.15 Firebase Cloud Messaging / Email — Notifications

Notifications help users stay updated about important activities on the platform. These may include order updates, price alerts, community interactions, and recommendations.
Why we chose it:

- Can notify users when an order status changes.
- Can provide alerts when a selected price condition is reached.
- Can notify users about new replies or discussions.
- Can send recommendation updates.
- Can be used for important administrative announcements.

1.16 Postman — API Testing

Postman is used to test the backend APIs independently from the frontend. This helps us verify whether an API is working correctly before connecting it to the user interface.
Why we chose it:

- Allows testing of GET, POST, PUT, and DELETE requests.
- Makes it easy to send JSON data to APIs.
- Helps check response data and HTTP status codes.
- Allows authentication headers to be tested.
- Helps identify and validate API errors.

1.17 Git and GitHub — Version Control

Git and GitHub are used to manage and track the project's source code. Since KrishiConnect involves multiple modules and potentially multiple team members, version control helps us work safely without losing previous changes.
Why we chose it:

- Keeps a history of project changes.
- Allows separate branches for different features.
- Makes it possible to restore earlier versions.
- Supports team collaboration.
- Provides remote backup of the source code.

1.18 Docker — Deployment and Environment Consistency

Docker can package the KrishiConnect application and its dependencies into separate containers. For example, the frontend, backend, and database-related services can be organized consistently.
Why we chose it:

- Reduces differences between development and deployment environments.
- Keeps project dependencies consistent.

- Makes deployment easier to repeat.
- Allows different services to be managed separately.

1.19 Cloud Deployment — Hosting

A cloud platform can be used to make KrishiConnect accessible outside the local development environment. The exact cloud provider can be selected based on the project's budget, available resources, and academic requirements. Why we chose it:

- Allows the project to be accessed remotely.
- Useful for project demonstrations and presentations.
- Provides centralized hosting for the application.
- Gives the system the possibility to scale as usage increases.

Data Flows

Product Listing Flow

Farmer Login
↓
JWT Authentication
↓
Farmer Dashboard
↓
Add Product Form
↓
Upload Product Image —→ Cloudinary/Firebase Storage
↓
FastAPI Product API
↓
Validation
↓
PostgreSQL Products Table
↓
Product Published
↓
Buyer Can Search/View Product

Order Flow

Buyer
↓
Search / Filter Products
↓
Product Details
↓
Purchase Request / Order
↓
FastA
PI
↓

Validate Buyer + Product + Quantity
↓
PostgreSQL Order Record
↓
Seller Notification
↓
Seller Accepts / Updates Status
↓
Buyer Notification
↓
Order History
↓
Rating / Review

Market Forecasting Flow

Historical Market Data
↓
Data Collection
↓
Pandas Data Cleaning
↓
Date / Commodity / Market Preparation
↓
ARIMA or Prophet
↓
Forecast Generation
↓
Forecast Evaluation
↓
Store / Serve Forecast Data
↓
Fast
API
↓
React Market Dashboard
↓
Charts + Forecast Information

Recommendation Flow

User Activity
↓
Searches / Views / Purchases / Topics
↓
UserActivity Data
↓
Feature Preparation
↓
Similarity / ML Recommendation Logic

↓
Candidate Products / Discussions
↓
Ranking / Filtering
↓
Recommendations API
↓
User Dashboard

QR Traceability Flow

Product Created
↓
Unique Product ID
↓
QR Code Generated
↓
QR Displayed / Printed
↓
Buyer Scans QR
↓
Traceability Endpoint
↓
Product Information Retrieved
↓
Product Details Shown

7. Expected Outcome

- The expected outcome is a working prototype of KrishiConnect that brings agricultural trading, market information, price forecasting, knowledge sharing, and intelligent recommendations into one platform.

- The system is intended to help farmers identify market opportunities, connect with buyers, understand price trends, and access useful agricultural information from a single, farmer-friendly platform.

8. Tentative Timeline

- September 2026 – project scoping, literature review, research-gap identification, requirements, and system planning.
- Late September to October 2026 – data preparation, architecture/database design, UI planning, and core marketplace development.
- October to November 2026 – price forecasting, market information, knowledge sharing, recommendation, ratings, notifications, and traceability modules.
- November 2026 – integration, testing, improvements, deployment, documentation, and final demonstration.

KrishiConnect

An Intelligent Platform for Agricultural Trade, Knowledge Sharing, and Market Insights

Empowering Farmers Through Data, Community and Opportunities

Farmers
Markets
Knowledge
Technology
Sustainable Future

1 USERS & ACCESS (Access from Anywhere)

- **Farmer** — List Products, Ask Questions, View Prices
- **Buyer** — Search Products, Place Orders, View Trends
- **Seller** — Manage Listings, Orders, Profile
- **Administrator** — Manage Users, Products, Reports
- Web | Mobile | Tablet (Responsive)
- Secure Access (HTTPS)

2 PRESENTATION LAYER (Frontend)

- React.js / Next.js
- HTML5, CSS3, JavaScript
- Tailwind CSS (Responsive UI)

Key User Interfaces

Login / Register · Dashboard · Product Listing · Market Charts · Community · Maps (Location) · Notifications · Profile

3 APPLICATION / API LAYER (Backend)

- FastAPI (Python)
- JWT Authentication
- Role-based Access Control

Microservices / Modules

User Service · Product Service · Order Service · Community Service · Market Service · Recommendation Service · Notification Service · Admin Service

4 DATA & MARKET PROCESSING

- **Pandas** (Data Cleaning)
- **NumPy** (Numerical Processing)

- Collect and integrate agricultural datasets
- Data cleaning and transformation
- Market-price data processing
- Feature engineering and preparation

5 INTELLIGENCE & MACHINE LEARNING

- **Scikit-learn** (Recommendation Engine)
- **ARIMA / Prophet** (Price Forecasting)

- Commodity price forecasting
- Personalized product recommendations
- User/product similarity analysis
- Model training, evaluation and optimization

6 DATABASE LAYER

- PostgreSQL (Reliable & Scalable)

Key Database Tables

Users · Products · Orders · Reviews · Questions · Comments · MarketPrices · Forecasts · Recommendations · Notifications · QRProducts · UserActivity

7 EXTERNAL SERVICES & INTEGRATIONS

- **Cloudinary / Firebase Storage** (Product Images)
- **OpenStreetMap + Leaflet** (Maps & Location)
- **QR Code Library** (Product Traceability)
- **Firebase Cloud Messaging / Email** (Notifications)
- **RAG (Optional)** Sentence Transformers + FAISS (Agri Knowledge Assistant)

8 CORE DATA FLOWS (Key Workflows)

- **Product Listing Flow** — Farmer lists product → Stored in DB → Visible to buyers
- **Order Flow** — Buyer places order → Order processing → Status updates
- **Market Forecasting Flow** — Market data → ML model → Price predictions
- **Recommendation Flow** — User activity → ML model → Personalized suggestions
- **QR Traceability Flow** — Scan QR code → Product information → Origin & traceability details

9 ANALYTICS & DECISION SUPPORT

- Market Price Trends (Interactive Charts)
- Product Analytics (Top Products, Demand)
- Forecast Visualizations (Future Price Trends)
- Recommendation Insights (User-based / Product-based)
- Admin Reports (Users, Orders, Activity)
- KPI Dashboard (Real-time Metrics)

10 DEPLOYMENT & SECURITY

- Docker (Containerization)
- Git / GitHub (Version Control)
- Cloud Platform (Deployment)
- HTTPS (Secure Access)
- Password Hashing & JWT Validation
- Input Validation & Error Handling
- Environment Variables & Config

REAL-TIME DATA FLOW

1. **User Action** (Search product / Place order / View market trends / Ask question)
2. **API Request** (Authenticated with JWT)
3. **PostgreSQL + Market Data** (Users, Products, Orders, Market Prices, etc.)
4. **ML / Forecasting Engine** (Recommendations, Price Prediction, Analytics)
5. **Results** (Products, Forecasts, (Recommendations, etc.)
6. **Notification / Dashboard** (Alerts, Updates, Visualizations)

KEY BENEFITS

- Integrated agricultural marketplace for farmers and buyers
- Transparent and real-time market insights
- Farmer knowledge sharing and community support
- Personalized recommendations using machine learning
- Product traceability with QR codes
- Secure, scalable and modern architecture
- Better decision-making for higher farmer income

"Grow Together, Trade Smarter"



KrishiConnect
An Intelligent Agri-Future

People | Data | Innovation | Sustainable Agriculture

Conclusion

KrishiConnect is designed as an integrated agricultural digital platform rather than only a product marketplace. React.js/Next.js provides the user interface, FastAPI provides the backend API and business logic, PostgreSQL manages structured application data, and Python data technologies provide the analytical and intelligent layer. Pandas and NumPy prepare market data, Scikit-learn supports intelligent recommendation functions, and ARIMA/Prophet provide agricultural price forecasting.

The core project features include user and role management, agricultural product trading, order management, knowledge sharing, market price analysis, forecasting, recommendations, notifications, dashboards, ratings, administration and QR traceability. Additional enhancements such as location-based search, price alerts, product comparison, advanced analytics, multilingual support and responsive mobile design can further extend the platform.

The proposed layered architecture separates presentation, APIs, business services, intelligence, data storage and external services. This separation makes KrishiConnect easier to develop, test, maintain and deploy, while providing a clear foundation for future improvements.